Show Your Work: Chain-of-Thought Scratchpads for Learned Modular Multiplication

Marcos Costa Santos Carreira — Team MCSC (MAC01-T00008) *SAIR Modular Arithmetic Challenge · methods note, rev. 2026-06-25*

> The naive question, the one a number theorist is too polite to ask out loud: if a transformer can be coaxed into modular *addition*, why does it fall over on `a · b mod p`? This note is the answer we backed into, and the one trick that moved the needle.

TL;DR. Feed-forward networks memorise per-prime multiplication tables and plateau (~0.5 on tier 2, floor on tier 3). An autoregressive **scratchpad** that generates the arithmetic as a chain-of-thought instead *learns the operation*: tier 2 to **0.98**, and — with an interleaved-Horner variant that never forms the full product — tier 3 to ~**0.82 on primes it never trained on** (0.79 / 0.82 / 0.85 across three seeds; held-out ≈ train, so it cannot be memorisation). A richer variant (an explicit reduce step + Abacus digit-position embeddings, with the training mix reweighted toward tier 3) then **clears the third tier outright**: on the official public benchmark it scores **tier-3 0.93 → `H>90 = 3`, overall accuracy 0.306** (deterministic, ~56 s for 1100 problems on CPU). So the progression is memorisation → generalisation (held-out ≈ train on unseen primes) → a *solved* third tier ($p < 10^5$), all from trained parameters with no hand-coded arithmetic. And the headline result is that the operation **scales**: the same recipe, only widened (WIDTH 5 → 10, one Horner step per digit), **clears all four tiers on held-out unseen primes** — tier 2 0.96, tier 3 0.93, tier 4 **0.905** ($p < 2^{32}$, ten-digit operands; `H>90 = 4` on the holdout). Generalisation carries across a doubling of operand length **nearly flat** — the harder ten-digit tier within ~0.02 of the five-digit one. A memorised table falls off a cliff the moment operands grow; a learned algorithm barely bends. That near-flat scaling, not the tier index, is the result. And it is now also the **live submission**: on the official evaluation hardware the same widened model clears all four scored tiers (tier 1 1.00, tier 2 0.97, tier 3 0.95, tier 4 **0.96**), `H>90 = 4`, overall accuracy **0.40**, in **5.1 s** for 1100 problems against a 300 s budget — so the result is not a research artefact held back by cost; it is fast, deterministic, and on the board. Widen the same recipe once more, to twenty-digit operands, and the operation still generalises — tier 5 **0.80** on the official benchmark — tracing a gentle curve (0.98 / 0.90 / 0.88 / 0.80 across 2 / 5 / 10 / 20 digits) that bends but never breaks. (That twenty-digit model's tier 5 sits below the 90% bar, so the ten-digit `H>90 = 4` model stays the submission; the wider run is the scaling evidence, not a leaderboard gain.)

The naive question

The task: emit `(a · b) mod p`, answer produced by trained parameters, no hand-coded arithmetic smuggled into the forward pass. Submissions are ranked by the highest tier cleared at ≥90% accuracy (`H>90`), then by mean accuracy. We reduce `a mod p` and `b mod p` legally before the network sees anything — two operands at a time, never the triple — so the learned problem is the clean one: `a, b ∈ [0, p)` ⟼ `a·b mod p`.

Everybody hits the same wall

We did the unglamorous part first — finding out what *doesn't* work, so the one thing that does has something to stand on.

A prime-conditioned feed-forward encoder (digits of `a%p, b%p, p` in, answer digits out) climbs to a tier-2 exact-match of ≈ 0.49 and **stops**. We then threw the standard levers at it, one at a time:

- **More parameters** (1.5M → 10M): the bigger model tracks the smaller one step-for-step. *Observation:* capacity is not the binding constraint.
- **Custom data distributions** — a low-/high-"wrap" mixture (Saxena et al. 2024) and two-set repeated-example training (Charton & Kempe 2024): ≈ 0 lift over uniform sampling at this scale.
- **Reciprocal / Barrett features** (handing the net `⌊10^{2W}/p⌋`): a sliver of unseen-prime traction, nothing on the larger primes.

Same fingerprint every time: drop in primes the model never trained on, and accuracy collapses to the floor. The network is memorising the 48 tier-2 multiplication tables, not learning the operation. This is *consistent with* Lauter, Li, Maughan, Newton & Srivastava (2024), who report both circular regression and a seq2seq transformer "demonstrate poor generalization" on exactly this problem.[1] When the people who set the exam tell you the obvious methods fail, that is a hint, not a dare.

Why the wall is there isn't mysterious. Peng, Narayanan & Papadimitriou (2024) prove, via communication complexity, that a bounded-depth Transformer cannot compose functions once their domains are large enough — and they name **multiplying multi-digit integers** as exactly the kind of sequential composition that fails, "failure growing with the required depth of composition."[2] A single forward pass simply lacks the depth, which is why adding parameters or data to a feed-forward model moves nothing. (A second, capacity-theoretic reason points the same way: Doshi, He, Das & Gromov (2024) show the closed-form network that solves modular *multiplication* needs width growing **exponentially**.) The scratchpad's contribution, below, is to supply the missing depth through *recurrence*: each autoregressive step is another effective layer, so the decode loop computes what one pass provably cannot.

Show your work

The fix is the oldest one in any maths classroom: make the model *show its work*.

Instead of a one-shot map to the answer, train a small causal decoder to generate the arithmetic as a chain-of-thought and read the answer off the end:

```
a · b = P ;  P ÷ p = k ;  P mod p = R          (R is the answer)
```

The worked steps `P, k, R` are **training data** — a teacher writing the algorithm on the board — not logic hard-coded into the forward pass. At inference the model generates `P, k, R` itself; randomise the weights and it emits noise. That is the whole compliance argument, and it is also the whole result.

One objection to disarm up front: doesn't worked-example supervision just *teach the algorithm in*? The steps are a teaching signal, not forward-pass logic — the model must still learn to produce each one — and the proof it *learned* rather than absorbed a lookup is that it generalises to primes it never saw (below). A compiled circuit would not need to be taught; a memoriser would not generalise.

[1] arXiv:2402.19254. Two of its authors are organisers of this challenge, which is the polite way of saying the bar was set by people who had already measured the wall.

[2] arXiv:2402.08164, *On Limitations of the Transformer Architecture*. The same paper makes the escape precise: chain-of-thought *can* dissolve the composition bound (their Theorem 2), but only if the generated trace spends a number of steps that **grows with the depth of the composition** — a single extra token is not enough. Our interleaved-Horner trace emits one reduction step per operand digit, so it spends exactly that budget: the method isn't just "use CoT," it is "use the amount of CoT the theorem demands." Recurrence is the cheaper realisation of the extra depth — we iterate rather than stack layers — and the one we take.

Observation. This is the first model in our sweep to break the 0.5 ceiling: tier-2 exact-match **0.98**, clearing the 90% threshold (`H>90 = 2`). The full 1100-problem set decodes in ≈ 42 s on CPU (budget: 300 s), deterministic.

Reversal of fortune: the part that isn't memorisation

The leaderboard number is nice; the number that matters is accuracy on primes **held out of training entirely** — where memorisation buys nothing. Reading the table requires care about *which model under which condition*, so the columns are labelled; the Horner tier-3 figure is the mean of three seeds (0.79 / 0.82 / 0.85):

Approach	tier-2, **unseen** primes	tier-3, **unseen** primes
reference `dlp_grokking`	≈ floor	≈ floor
feed-forward encoder	0.13	≈ 0.05
digit scratchpad (`P, k, R`)	0.35	— (can't form the 10-digit product)
interleaved-Horner scratchpad	**0.83**	**0.82 ± 0.03 (3 seeds; held-out ≈ train)**

Two reads. (1) Each scratchpad step improves *generalisation* — the digit scratchpad already triples the memorising encoder on unseen tier-2 primes (0.35 vs 0.13); the Horner variant, which never forms the full product, carries it to tier 3. (2) The **held-out ≈ train** equality at tier 3 is the load-bearing evidence: tier 3 draws from thousands of primes, so a 0.82 it can't have memorised. *Observation, not theorem:* the model has learned something closer to the operation than the table — and it *appears* the chain-of-thought is doing the work, since the same architecture without the worked steps is the seq2seq baseline that does not generalise (Lauter et al.).

Provenance of the headline numbers (so nothing is mistaken for cherry-picking): 0.83 / 0.82 are *held-out, truly-unseen* estimates from the Horner model (the tier-3 0.82 is the three-seed mean, 0.79 / 0.82 / 0.85) — the *generalisation* claim. The tier-3 **no-holdout** model (`recurrent-v3-ship`, the `H>90 = 3` predecessor to the live tier-4 submission) instead *fits* the whole in-band prime set and is scored on the official public benchmark (100 problems/tier): tier 1 1.00, tier 2 0.99, **tier 3 0.93**, overall accuracy **0.306**, `H>90 = 3`, deterministic. So the unseen-prime result (0.82) is the evidence the model learned the *operation*, and the no-holdout submission turns that learned operation into a cleared tier on the benchmark. One honest margin note: tier-3 0.93 on a 100-problem tier sits ~1.2 standard errors above the 0.90 line, so on a different random draw (the private final eval) it could land nearer the bar; the generalisation result is the more robust claim.

One honest caveat that cuts both ways: He, Doshi, Das & Gromov (2024) show out-of-distribution generalisation in modular arithmetic is **transient** — it emerges then decays. That is *why* we select checkpoints by held-out accuracy (to catch the peak). The three seeds reported here (0.79 / 0.82 / 0.85, mean 0.82 ± 0.03) were each still *rising* at the 5 h budget cap, so these are peak-of-budget point estimates — if anything conservative, not converged ceilings.

> **Ask the bot, ignore the answer.** The first shipping run peaked at tier-2 0.885 and then *drifted down* while the training loss kept falling — the textbook "more steps must be better" intuition, quietly wrong. The learning rate was too hot late; a gentle fine-tune at 3e-5 from the peak walked it through 0.90 to 0.98 in 2,500 steps. The model was done learning the operation and had started over-fitting the teacher-forcing objective. Watch the held-out curve, not the loss.

The long way around: never build the product (tier 3)

Tier 3 (p up to ~65k) wants an exact 10-digit product, and the basic scratchpad chokes on generating it — 5×5 long multiplication, every digit and carry exact. So we don't build it. Process `b` one digit at a time, Horner-style, reducing as we go:

```
acc ← 0
for each digit b_i of b, most-significant first:
    acc ← (acc·10 + a·b_i) mod p
R ← acc        # the last acc is the answer
```

The full product never exists. Every step is small — multiply `a` by a single digit (≤ 6 digits), shift, add, reduce mod `p` — and the procedure is identical for every prime, which is the property memorisation can never have.

One worked example (`a = 200, b = 137, p = 251`; answer `27400 mod 251 = 41`):

digit `b_i`	`m = a·b_i`	`acc = (acc·10 + m) mod 251`
0	0	0
0	0	0
1	200	200
3	600	(2000 + 600) mod 251 = 90
7	1400	(900 + 1400) mod 251 = **41**

Five small steps, no 10-digit anything. The model emits each `m` and running `acc`; the last `acc` is the answer. Trained with an operand-length curriculum (short products first, grown to full).

Result. It works, and it generalises. On tier-3 primes **held out of training entirely**, the Horner model reaches exact-match **0.82 ± 0.03 across three seeds (0.79 / 0.82 / 0.85), with held-out ≈ train** — the equality is the point: tier 3 draws from thousands of primes, so a 0.82 it can't have memorised and can't have compiled. It learned the *operation* and ran it on operands and primes it never saw. Every prior approach in our sweep — the feed-forward encoder, capacity, custom data distributions, reciprocal features — sat at the ~0.05 floor on tier 3, unable even to *fit* it in-sample. The curriculum is load-bearing: tier-3 accuracy climbs 0.00 → 0.31 → 0.62 → ~0.82 as the operand cap grows from 1 to 5 digits.

Clearing the tier: a richer scratchpad (the capstone)

Generalising to ~0.82 on unseen primes shows the model learned the operation; it does not yet *clear* the 90% bar. Two changes close that last stretch, both aimed at the exactness of the long-multiplication carry rather than at memorising more:

- **An explicit reduce step.** The Horner trace emits `t = acc·10 + m` as its own token group before reducing `t mod p`, so the network is supervised on the intermediate it previously had to form implicitly.
- **Abacus digit-position embeddings** (McLeish et al. 2024): position-of-digit embeddings that sharpen carry alignment — the same fix the position-shift literature (Yudin 2026) recommends for text-based arithmetic.

Trained no-holdout with the learning rate annealed to zero at full length and the training mix **reweighted toward tier 3** (so capacity is spent where the bar is contested, not over-perfecting an already-solved tier 2), the model clears it. On the official public benchmark (100 problems/tier): tier 1 **1.00**, tier 2 **0.99**, tier 3 **0.93** — `H>90 = 3`, overall accuracy **0.306**, deterministic, ~56 s for 1100 problems on CPU (KV-cached generation, well inside the 5-minute budget). The same recurrence argument carries through: randomise the weights and it emits noise; the answer is computed, step by step, by trained parameters.

Same operation, more digits: the scaling is the result

The sharpest result here is not a cleared tier — it is that the *operation scales*, and that is the thing a memorised table can never do. Enlarge the operands and a lookup falls to the floor: that is exactly what the feed-forward encoder does, and what the hand-built carry circuits of the "trivial" file do past five digits. A learned procedure scales instead — and this one does, **by construction**. Going from tier 3 ($p < 10^5$, $\leq$ 5-digit operands) to tier 4 ($p < 2^{32}$, $\leq$ 10-digit operands) changes nothing about the method: same legal reduce-first, same `a · digit` Horner that never forms the product. We only widen the digit fields (WIDTH 5 $\rightarrow$ 10) and run more steps — **one Horner step per digit**. The cost model is linear in the operand length, which is precisely the composition *depth* the Peng–Narayanan–Papadimitriou bound says the task needs; the recurrence supplies that depth, and supplies more of it automatically as the operand grows.

And what scales is the property that matters — generalisation to primes never trained on. The widened model (WIDTH = 10, base-10 `a · digit` Horner, ten steps) **clears all four tiers on held-out, unseen primes**, with almost no loss across the operand-length doubling:

held-out, **unseen** primes	tier 2	tier 3 ($\leq$ 5-digit)	tier 4 ($\leq$ 10-digit)
WIDTH-10 interleaved-Horner scratchpad	0.96	**0.93**	**0.905**

(Single seed; converged over a finishing LR anneal; tier-4 held 0.89–0.93 across eight evals, mean $\approx$ 0.905.) Read that table again: the *harder* tier — ten-digit operands, primes drawn from billions — generalises within ~0.02 of the five-digit tier. The operation does not just survive a doubling of operand length, it carries across it nearly flat. That is far past "graceful degradation": a memorised table falls off a cliff the instant the operands grow; this **bends so little it barely registers** — the single strongest piece of evidence in this note that the model learned the *operation*, not a per-prime table. The only changes from the tier-3 model are wider digit fields and more Horner steps — no new hand-coded arithmetic, the answer still the model's own generated final `acc`.

On the official benchmark, not just the holdout. The same widened model was then scored by the challenge's own evaluation on its own hardware (1100 problems, 100 per tier): tier 1 **1.00**, tier 2 **0.97**, tier 3 **0.95**, tier 4 **0.96** — `H>90 = 4`, overall accuracy **0.40**, deterministic. Because tiers 3–4 draw their primes from thousands-to-billions, the benchmark primes are effectively unseen to the model, so this official number *is* the generalisation result, now independently confirmed off our own holdout. And it is cheap: the full 1100-problem set decodes in **5.1 seconds** against the 300-second budget — roughly sixty times under. (A cautionary aside for anyone reproducing this: an `x86`-emulated local Docker run inflated the wall-clock by two orders of magnitude and looked like a timeout; the real hardware ran it in seconds. Measure on the target environment, not an emulated proxy.) The efficiency axis the organisers floated — score speed, not just accuracy — this submission already satisfies.

One tier further, to twenty digits. We then widened the *same* recipe once more — the single literal `WIDTH 10 → 20`, nothing else — to reach tier 5 ($p < 2^{64}$, ≤ 20-digit operands, twenty Horner steps). Warm-started from the tier-4 model and run through the same curriculum-then-anneal, it scores on the official benchmark:

official benchmark	tier 2	tier 3 (≤5d)	tier 4 (≤10d)	tier 5 (≤20d)
WIDTH-20 scratchpad	0.98	0.90	0.88	**0.80**

(One run; 53.5 s for 1100 problems — about ten times the WIDTH-10 model, still a fraction of the 300 s budget.) The operation keeps generalising to unseen primes a *full doubling* of operand length past where the tier-4 model stopped: twenty-digit products, primes drawn from quintillions, and it still lands four in five. The curve across $2 \to 5 \to 10 \to 20$ digits — 0.98, 0.90, 0.88, 0.80 — slopes off by only ~0.18 end to end and never falls off the cliff a lookup would the instant operands grow. That gentle decline, traced by *one* algorithm over a sixteen-fold range of operand magnitude, is the scaling claim in its strongest form.

Two honest qualifications. First, tier 5's 0.80 is *below* the 90% bar, so this model's frontier index is `H>90 = 3`; the tier-4 model remains the leaderboard submission. Second, this twenty-digit model's tier-3 and tier-4 (0.90, 0.88) sit slightly *under* the ten-digit model's (0.95, 0.96): spreading a fixed ~11M-parameter budget across five tiers of longer sequences costs some per-tier sharpness. So the claim is precisely "the same recipe *reaches* twenty digits with graceful degradation," not "wider is uniformly better." The scaling is real, and it is gentle — but it is not free.

The competition scores this as a tier index (`H>90 = 4`). What it *is* is a learned, generalising, length-scalable algorithm for modular multiplication — and the scaling, not the index, is the result. *(Earlier we said the radix could be raised to cut the step count; it cannot — a base-1000 chunk makes each step a 3-digit × 3-digit multiply the network never learns, stalling tier 2 at 0.34. The trivial `a · digit` step is load-bearing: per-step learnability dominates step-count. That negative is itself part of the result.)*

A note on the "it's trivial" file

Mid-competition, a strong entrant argued the task is trivial and posted hand-built networks: long multiplication, then long division, in ML ops.[3] Useful to engage, because they sharpen what the contribution is.

[3] Public gists, kept anonymous here. The author is candid that they are handcrafted circuits and that the differentiable variant fails past ~5 digits; the point below is that this *strengthens* the case for a learned, iterated model rather than weakening it. Both are **handcrafted circuits** — the author says so — with weights set by hand (`9`, `10`), zero trained parameters; that is the prohibited construction, and it answers the *boring* question (is `a·b mod p` *representable* as a network? yes, trivially, since Jane Street's MD5-as-ReLU). The interesting concession is in the fine print: *"a trained carry will fail after 5 digits; fixing this by iterating the carry would make this demonstration more complex, so I left it like that."* That is exactly the gap. A trained, **continuous** carry breaks past five digits unless you **iterate** — and the stated obstruction ("everything must be continuous for gradients") binds only a single-pass differentiable circuit. The scratchpad sidesteps it: we never backpropagate through a discrete carry; each carry and reduction is a **token-level classification target**, learned by teacher forcing, and the autoregressive decode *is* the iteration. So the model learns the very thing the construction concedes a trained network cannot — and generalises across primes it never saw. Representability was never the question; learning was.

Where this stands in the literature

Restraint first: the components here are borrowed; the contribution is their composition and the result.

The terrain splits, and this work is on the hard side of it. Modular **addition** is largely understood — grokking and clean Fourier / closed-form circuits (Power et al. 2022; Gromov 2023; Zhong et al. 2023; McCracken et al. 2025); addition mod p is a rotation that networks learn and that interpretability can fully open. None of that is this problem. Modular **multiplication** is the recognised hard case, and the directly relevant precursor is the challenge organisers' own paper: Lauter, Li, Maughan, Newton & Srivastava (2024) tried circular regression and a one-shot seq2seq transformer and reported poor generalisation — read as evidence of the task's hardness. The delta here is exactly the change of method: a one-shot map is depth-starved (Peng, Narayanan & Papadimitriou 2024 — bounded depth cannot compose), and the autoregressive scratchpad supplies the missing depth, turning their negative into a positive (held-out ≈ train across tiers).

Two precisions keep the claim honest. First, this is **not** "the first network to learn modular multiplication": Charton & Kempe (2024, repeated examples) and Saxena et al. (2024, custom data, an LWE setting) already learn it *in-distribution* at fixed or structured moduli. The sharper delta is **cross-prime generalisation** — the operation transferring to moduli never trained on, the challenge's actual open question, where the seq2seq baseline fell short — and the **scaling** of that generalisation across operand length. Second, the scratchpad idea (Lee et al. 2023; Nye et al. 2021), the Abacus digit-position embeddings (McLeish et al. 2024), the operand curriculum, and the depth-limit theory are all off the shelf; the novelty is the assembly and the result, not the parts.

Stated with the restraint it deserves: this *appears* to be a positive cross-prime-generalisation-and-scaling result for learned modular multiplication, where the organisers' own baseline was negative, with the depth-versus-recurrence argument as the mechanism. The freshest single angle is the scaling-as-discriminator — graceful degradation across a doubling of operand length as the signature of a learned operation rather than a memorised table — a framing not, as far as we know, applied to modular multiplication before, though length generalisation for *non-modular* integer arithmetic is established (McLeish et al.).

Why this wasn't done before. The approach sits in the gap between two communities that did not overlap. The modular-arithmetic-learning line came from cryptography / math-ML (Lauter; Charton & Kempe; Saxena; the LWE attacks) and used one-shot seq2seq or regression — and for a *crypto-attack* question a taught scratchpad is beside the point: you want the network to learn the map black-box, so handing it the worked steps undercuts the very thing being tested. The chain-of-thought line, meanwhile, came from LLM reasoning (Nye et al.; Lee et al.) and was aimed at word problems and ordinary integer arithmetic, not modular multiplication or transfer across moduli. Nobody pointed one at the other. What changed is the competition's framing: it makes a *learned-from-parameters* scratchpad legitimate (the answer is computed, not looked up) and asks specifically for *generalisation to unseen primes* — the exact question the scratchpad answers, and one you would not even instrument (hold out primes, measure cross-modulus transfer) without that framing. The enabling parts — the depth-limit result, Abacus embeddings, the CoT-arithmetic recipes — only aligned recently. We claim no more than this: the incentives and the framing pointed elsewhere, and we are not aware of prior work assembling these pieces for cross-prime modular-multiplication generalisation and its scaling.

Limitations, and what would make this a paper

What this is **not**, plainly:

- **Not mechanistic.** The addition literature exhibits the learned circuit; here we show behaviour, not internals. A probe / activation-patching pass to localise what the scratchpad computes is the single biggest upgrade.
- **Not a theorem.** An empirical existence-and-scaling result; no guarantee the scratchpad generalises in general.
- **Small-scale relative to the motivation.** The cryptographic line behind this work (Lauter; Saxena; SALSA, Wenger et al. 2022) concerns hundreds of digits and structured LWE; this reaches twenty (tier 5, $p < 2^{64}$). The contribution is to the learnability/generalisation science, not to anything cryptographic.
- **Uneven rigour across tiers.** Tier 3 carries a three-seed band (0.82 ± 0.03); the WIDTH-10 tier-4 result (`H>90 = 4`, official 0.96) and the WIDTH-20 tier-5 result (0.80) are each a single converged run. The twenty-digit run also shows the honest cost of reach: its tier-3/4 (0.90/0.88) fall a little below the ten-digit model's (0.95/0.96), and its tier-5 0.80 is below the 90% bar — so it is the scaling evidence, not a leaderboard improvement over the ten-digit model.

The checklist to turn this from a competition result into a citable paper:

1. **Multi-seed tier 4** — match the tier-3 rigour; report a band, not a point.
2. **Radix ablation** — base-10 vs base-100 vs base-1000, already half-run: the base-1000 failure (the multi-digit per-step multiply is unlearnable — per-step *learnability* beats step-count) is itself a finding worth stating cleanly.
3. **A look inside** — even a linear-probe confirmation that the intermediate `acc` tokens carry the running reduction lifts it from black-box to mechanism-adjacent.
4. **A same-compute baseline head-to-head** — seq2seq vs scratchpad — to *quantify* the depth-versus-recurrence claim rather than inherit it from Lauter's report.
5. **Push the scaling curve** — *partly done*: tier 5 (twenty digits) reads 0.80 on the official benchmark, so the curve is 0.98 / 0.90 / 0.88 / 0.80 across 2 / 5 / 10 / 20 digits and is bending, not breaking. What remains is (a) multi-seed bands at each width, (b) more capacity (the fixed ~11M budget, spread across five tiers of longer sequences, is what pulls tier 5 below the bar — a larger model is the test of whether the slope is fundamental or just capacity-limited), and (c) tier 6+ to find where linear-depth finally meets the fidelity wall.

The fine print (compliance)

Answer from trained parameters (weight perturbation collapses it). Only per-argument reductions `a%p`, `b%p`; the scratchpad steps are training targets, not a by-construction algorithm in the forward pass; no symbolic-math libraries, no hand-coded long multiplication/division, no Barrett/Montgomery, no CRT, no input-indexed lookups at inference. Deterministic on CPU.

A note on format robustness, since text-based arithmetic models are known to be brittle to it: Yudin (2026) documents a "position-shift cliff" — character-level modular-arithmetic models that hit high in-distribution accuracy then collapse when the same expression is moved to different absolute positions or templates. Our design sidesteps it by construction. We reduce `a%p, b%p` first and emit **fixed-width fields**, so every operand and the answer sit at the same offset regardless of prime or tier; the model is never exposed to the positional variance that breaks the baseline. (The

richer v3 variant goes further, adding Abacus digit-position embeddings — precisely the position-aware alignment Yudin recommends.)

Reproducibility

Argument-free runners (`run_*.py`) in the team repo; the scored submission is the HuggingFace model `MCarreira/recurrent-v4` @ commit `467ad5d7180bbfae21b55185cfeca22e2e18b458` (the WIDTH-10 `H>90 = 4` model above; its tier-3 predecessor `MCarreira/recurrent-v3-ship` reaches `H>90 = 3`, and the WIDTH-20 scaling model `MCarreira/recurrent-v5` reaches tier 5 0.80 / `H>90 = 3`); trainer `dev/train_recurrent.py`; standard `ModularMultiplicationModel` interface; full experiment log in `EXPERIMENTS.md`. Every number here is reproduced by the official local pipeline (`modchallenge evaluate` / `evaluate-hf`) — nothing is a fabricated parameter.

AI-assistance acknowledgement: experiments, code, and a first draft of this note were produced with the assistance of a large language model (Claude), under the author's direction and verification.

References

- Peng, Narayanan, Papadimitriou. *On Limitations of the Transformer Architecture.* arXiv:2402.08164.
- Lauter, Li, Maughan, Newton, Srivastava. *Machine Learning for Modular Multiplication.* arXiv:2402.19254.
- Saxena, Alfarano, Charton, Allen-Zhu, Wenger, Lauter. *Making Hard Problems Easier with Custom Data Distributions and Loss Regularization.* arXiv:2410.03569.
- Charton, Kempe. *Emergent Properties with Repeated Examples.* arXiv:2410.07041.
- He, Doshi, Das, Gromov. *Learning to grok…* arXiv:2406.02550.
- Doshi, He, Das, Gromov. *Grokking Modular Polynomials.* arXiv:2406.03495.
- Yudin. *Mitigating Position-Shift Failures in Text-Based Modular Arithmetic via Position Curriculum and Template Diversity.* arXiv:2601.04283.
- McLeish et al. *Transformers Can Do Arithmetic with the Right Embeddings (Abacus embeddings).* arXiv:2405.17399.
- Lee et al. *Teaching Arithmetic to Small Transformers.* 2023.
- Nye et al. *Show Your Work: Scratchpads for Intermediate Computation with Language Models.* 2021.
- Power, Burda, Edwards, Babuschkin, Misra. *Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets.* arXiv:2201.02177.
- Gromov. *Grokking Modular Arithmetic.* arXiv:2301.02679.
- Zhong, Liu, Tegmark, Andreas. *The Clock and the Pizza: Two Stories in Mechanistic Explanation of Neural Networks.* arXiv:2306.17844.
- Li, Liang, Shi, Song, Zhou. *Fourier Circuits in Neural Networks and Transformers: A Case Study of Modular Arithmetic with Multiple Inputs.* arXiv:2402.09469.
- McCracken, Moisescu-Pareja, Létourneau, Precup, Love. *Uncovering a Universal Abstract Algorithm for Modular Addition in Neural Networks.* arXiv:2505.18266.
- Wenger, Chen, Lauter, Charton. *SALSA: Attacking Lattice Cryptography with Transformers.* arXiv:2207.04785.